

Original code

github repo: <https://github.com/smn-krn/SBD3-Part2/tree/main/Exercise3>

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, from_json, lower, count, desc
from pyspark.sql.types import StructType, StructField, StringType, LongType
...
```

The original implementation represents a **baseline Structured Streaming pipeline** whose main goal was to demonstrate Kafka ingestion and basic aggregation. Its logic can be summarized as follows:

- The stream was filtered on the substring "**vulnerability**" inside the **content** field.
- Only records with severity "**High**" were considered.
- Aggregation was performed by grouping on **source_ip**.
- The aggregation produced a global count without any notion of time windows.
- The computation implicitly relied on **processing-time semantics**, meaning that results depended on when Spark processed the records rather than when the events actually occurred.

From a technical perspective, this approach is simple and effective for **static or exploratory analysis**, but it has important limitations for real-time monitoring scenarios:

- There is no temporal context: spikes or bursts cannot be isolated in time.
- Late-arriving events cannot be handled correctly.
- Aggregation by **source_ip** does not reflect user-level behavior.
- The lack of windowing means state can grow indefinitely in long-running streams.

As a result, the original code is not suitable for **near real-time user experience monitoring**, where correctness depends on event timestamps and bounded aggregation intervals.

Modified code (Activity 3 implementation)

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    col,
    from_json,
    lower,
    count,
    window,
    to_timestamp
)
from pyspark.sql.types import (
    StructType,
    StructField,
    StringType,
    LongType
)
```

```
CHECKPOINT_PATH = "/tmp/spark-checkpoints/activity-3"

spark = (
    SparkSession.builder
        .appName("Activity3-CrashMonitoring")
        .config("spark.sql.streaming.checkpointLocation", CHECKPOINT_PATH)
        .getOrCreate()
)

spark.sparkContext.setLogLevel("ERROR")

schema = StructType([
    StructField("timestamp", LongType()),
    StructField("status", StringType()),
    StructField("severity", StringType()),
    StructField("source_ip", StringType()),
    StructField("user_id", StringType()),
    StructField("content", StringType())
])

raw_df = (
    spark.readStream
        .format("kafka")
        .option("kafka.bootstrap.servers", "kafka:9092")
        .option("subscribe", "logs")
        .option("startingOffsets", "earliest")
        .option("failOnDataLoss", "false")
        .load()
)

parsed_df = (
    raw_df
        .select(from_json(col("value").cast("string"), schema).alias("data"))
        .select("data.*")
)

events_df = parsed_df.withColumn(
    "event_time",
    to_timestamp(col("timestamp"))
)

filtered_df = (
    events_df
        .filter(lower(col("content")).contains("crash"))
        .filter(col("severity").isin("High", "Critical"))
)

aggregated_df = (
    filtered_df
        .withWatermark("event_time", "20 seconds")
        .groupBy(
            window(col("event_time"), "10 seconds"),
            col("user_id")
        )
)
```

```
    )
    .agg(count("*").alias("crash_count"))
    .filter(col("crash_count") > 2)
  )

query = (
  aggregated_df
  .writeStream
  .outputMode("append")
  .format("console")
  .option("truncate", "false")
  .start()
)

query.awaitTermination()
```

The modified implementation fundamentally changes the **semantics, correctness guarantees, and analytical value** of the streaming application.

Instead of focusing on infrastructure-level signals (such as IP addresses), the pipeline now targets **user-centric crash detection**, which is more meaningful for monitoring user experience and application stability.

Key improvements include:

- Transition from processing-time to **event-time semantics**
- Introduction of **time-bounded aggregation**
- Explicit handling of **late-arriving events**
- User-level aggregation instead of infrastructure-level aggregation

Event-time & Windowing

The application now operates on **event time**, derived from the **timestamp** field embedded in each log record. This timestamp represents when the event actually occurred at the source, not when it was ingested or processed by Spark.

A **10-second tumbling window** is applied:

```
window(col("event_time"), "10 seconds")
```

This design choice ensures that:

- Each event contributes to exactly one window
- Aggregations are strictly time-bounded
- Results reflect real temporal behavior rather than processing delays

By grouping on both **user_id** and the event-time window, the system can detect **short-term bursts of crash events per user**, which is essential for near real-time alerting.

Watermark & Late Data

To handle out-of-order and delayed events, a watermark of **20 seconds** is defined:

```
.withWatermark("event_time", "20 seconds")
```

This watermark specifies how long Spark should wait for late-arriving data before finalizing a window.

Technically, this achieves two critical goals:

- **Correctness:** events that arrive slightly late are still included in the correct window
- **State management:** Spark can safely drop old state, preventing unbounded memory growth

Events arriving later than the watermark are discarded, which represents a deliberate trade-off between completeness and system stability. In real-world streaming systems, such trade-offs are necessary to ensure predictable resource usage.

Scalability & Fault Tolerance

The final design satisfies the non-functional requirements of scalability and fault tolerance:

- **Kafka partitions** allow parallel consumption across executors
- **Spark executors** can scale horizontally to increase throughput
- **Checkpointing** persists offsets and aggregation state
- **Stateful windowed aggregations** are fully recoverable after failures

If a worker or executor crashes, Spark can restart the computation and restore the exact state of the streaming query using the checkpoint directory, ensuring **at-least-once processing semantics**.

Re-execution

```
spark-submit \  
  --master spark://spark-master:7077 \  
  --packages org.apache.spark:spark-sql-kafka-0-10_2.13:4.0.0 \  
  --num-executors 1 \  
  --executor-cores 1 \  
  --executor-memory 1G \  
  /opt/spark-apps/spark_structured_streaming_logs_processing.py
```

and then

```
docker compose up -d
```

← → ↺

http://localhost:8080

Anmelden

spark

4.0.0

Spark Master at spark://f49459b969df:7077

URL: spark://f49459b969df:7077

Workers: 1 Alive, 0 Dead, 0 Decommissioned, 0 Unknown

Cores in use: 1 Total, 1 Used

Memory in use: 1024.0 MiB Total, 1024.0 MiB Used

Resources in use:

Applications: 1 Running, 3 Completed

Drivers: 0 Running (0 Waiting), 0 Completed (0 Killed, 0 Failed, 0 Error, 0 Relaunching)

Status: ALIVE ([Environment](#), [Log](#))

▼ Workers (1)

Worker Id	Address	State	Cores	Memory
worker-20260120192959-172.20.0.5-37533	172.20.0.5:37533	ALIVE	1 (1 Used)	1024.0 MiB (1024.0 MiB Used)

▼ Running Applications (1)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20260120195341-0003	(kill) Activity3-CrashMonitoring	1	1024.0 MiB		2026/01/20 19:53:41	root	RUNNING	51 s

▼ Completed Applications (3)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20260120194748-0002	LogsProcessor	0	2.0 GiB		2026/01/20 19:47:48	root	FINISHED	2.9 min
app-20260120194601-0001	LogsProcessor	0	2.0 GiB		2026/01/20 19:46:01	root	FINISHED	1.7 min
app-20260120193726-0000	LogsProcessor	1	1024.0 MiB		2026/01/20 19:37:26	root	FINISHED	4.7 min

← → ↺

http://localhost:4040/stages/

Anmelden

spark

4.0.0

Jobs

Stages

Storage

Environment

Executors

SQL / DataFrame

Structured Streaming

Activity3-CrashMonitoring application UI

Stages for All Jobs

Active Stages: 1

Completed Stages: 3

▼ Active Stages (1)

Page: 1

1 Pages. Jump to 1 . Show 100 items in a page. Go

Stage Id	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
3	id = 6caa0e1a-5579-477f-8c1c-25d778e88593 runId = 3487c4b5-de2c-4ca9-af8c-9dab07d905c9 b... start at <unknown>:0 +details (kill)	2026/01/20 19:55:11	8 s	104/200 (2 running)			44.7 KiB	

Page: 1

1 Pages. Jump to 1 . Show 100 items in a page. Go

▼ Completed Stages (3)

Page: 1

1 Pages. Jump to 1 . Show 100 items in a page. Go

Stage Id	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
2	id = 6caa0e1a-5579-477f-8c1c-25d778e88593 runId = 3487c4b5-de2c-4ca9-af8c-9dab07d905c9 b... start at <unknown>:0 +details	2026/01/20 19:55:07	4 s	2/2				86.1 KiB
1	id = 6caa0e1a-5579-477f-8c1c-25d778e88593 runId = 3487c4b5-de2c-4ca9-af8c-9dab07d905c9 b... start at <unknown>:0 +details	2026/01/20 19:54:45	21 s	200/200			538.5 KiB	
0	id = 6caa0e1a-5579-477f-8c1c-25d778e88593 runId = 3487c4b5-de2c-4ca9-af8c-9dab07d905c9 b... start at <unknown>:0 +details	2026/01/20 19:53:49	56 s	2/2				538.5 KiB

Page: 1

1 Pages. Jump to 1 . Show 100 items in a page. Go